Task 1: German Tank Problem

COURSE NAME

CSDS 413 Introduction to Data Analysis

Authors:

Jacob Anderson

October 13, 2025

Contents

Context

Recall the German Tank (or the “Rocket Science") problem from Exercise #7 in the class. You know that the Nazi Army is assigning sequential integer IDs to their tanks. In other words, if they have M tanks, there is a tank with ID i for all $1 \leq i \leq M$. The allied forces capture n tanks and the IDs of these tanks are $1 \leq x_1 \leq x_2 \leq ... \leq x_n$. Our goal is to estimate M, i.e., the number of tanks that the Nazis have, based on the assumption that the tanks were captured uniformly at random from these M tanks.

1 Part A

Let $\hat{M}_{MLE}$ be the maximum likelihood estimator for M. Compute $\hat{M}_{MLE}$.

—

Under the assumption in the context, "the tanks were captured uniformly at random from these M tanks", we are looking at some unique list of n tank IDs from $\binom{M}{n}$ possible observations, making the probability of an individual combination of n tank IDs captured $\frac{1}{\binom{M}{n}}$.

However, the likelihood of this observation is non-zero under the condition that M is greater than or equal to x_n, or else it would be impossible to observe x_n under this capturing assumption:

$$L(M) = \begin{cases} \frac{1}{\binom{M}{n}} & \text{if } M \geq x_n \\ 0 & \text{if } M < x_n \end{cases}$$

Regarding the maximum, likelihood is zero for M less than x_n, and when M is greater than x_n, $\binom{M}{n}$ becomes larger and makes our specific observation of IDs less likely given a random, uniform sampling, meaning the likelihood must decrease for increasing M, so the MLE estimator must take on the most constraining value of M that does not make the likelihood of the observation impossible, which is the maximum observed ID: $\hat{M}_{MLE} = x_n = \max(x_1, x_2, \ldots, x_n)$.

Below is the implementation of the estimator for M:

```
def tank_capture(M: int, n: int) -> list:
    """
    Capture n tank IDs uniformly at random from M total tanks.

    :param M: Total number of tanks
    :type M: int
    :param n: Number of tanks captured
    :type n: int
    :returns: List of captured tank IDs
    :rtype: list
    """
    return sorted(np.random.choice(range(1, M + 1), size=n, replace=False).tolist())

def mle_M(tank_ids: list) -> int:
    """
    Computes the MLE for the total number of tanks given
    an observation of tank IDs.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: Maximum likelihood estimate of total tanks
    :rtype: int
    """
    return max(tank_ids)
```

Figure 1: Problem setup and MLE computation.

Here is an arbitrary example of $(M = 100, n = 36)$, just to exemplify the computation:

```
(assignment2-py3.12) jande@BT-7274:~/csds413_assignments/
assignment2/task1_german_tank/scripts$ python
Python 3.12.0 (main, Jun  9 2025, 09:52:12) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from estimators import mle_M
>>> from sim import tank_capture
>>> captured_ids = tank_capture(100, 36)
>>> print(f"Captured tank IDs: {captured_ids}")
Captured tank IDs: [1, 4, 8, 9, 10, 12, 16, 18, 19, 20, 27, 28, 31,
43, 44, 45, 50, 52, 56, 58, 61, 62, 65, 68, 70, 71, 72, 78, 79, 81,
83, 84, 86, 94, 95, 97]
>>> print(f"MLE of total tanks: {mle_M(captured_ids)}")
MLE of total tanks: 97
```

2 Part B

Let $\hat{M}_{MEAN} = 2(\sum_{i=1}^{n} x_i/n) - 1$ and $\hat{M}_{MVU} = x_n(\frac{n+1}{n}) - 1$ be two additional candidate estimators for M. Theoretically characterize the expected value of each of the three estimators ($\hat{M}_{MLE}$, $\hat{M}_{MEAN}$ $\hat{M}_{MVU}$) for a given M and N. Comment on the bias of each estimator.

—

These expected values for each estimator were given in the recent Canvas announcement:

$$
\begin{aligned}
E[\hat{M}_{MLE}] &= \frac{n}{n+1}(M+1) \\
E[\hat{M}_{MEAN}] &= M \\
E[\hat{M}_{MVU}] &= M
\end{aligned}
$$

Generally speaking, the bias of an estimator is the difference between the expected value of the estimator and the true parameter value. That makes $E[\hat{M}_{MEAN}] = M$ and $E[\hat{M}_{MVU}] = M$ very straightforward cases: $M - M = 0$; these two have no bias, they are unbiased estimators. The MEAN estimator correctly estimates M on average by using the sample mean of observed tank IDs because under uniform sampling, the average captured ID value should approximate the true mean of the tank population. The MVU estimator achieves M on average by applying $(n+1)/n$ to the maximum observed value as a correction factor to the undervaluation by the MLE.

Regarding the bias of $\hat{M}_{MLE}$ as $E[\hat{M}_{MLE}] - M = \frac{n}{n+1}(M+1) - M$, this can be simplified to $-\frac{M-n}{n+1}$:

$$
\begin{aligned}
E[\hat{M}_{MLE}] - M &= \frac{n}{n+1}(M+1) - M \\
&= \frac{n(M+1) - M(n+1)}{n+1} \\
&= \frac{nM + n - Mn - M}{n+1} \\
&= \frac{n - M}{n+1} \\
&= -\frac{M-n}{n+1}
\end{aligned}
$$

The bias is negative, so on average the MLE will underestimate the total number of tanks. That reflects well how the MLE computation works, because the only cases where MLE is M are the cases where the random, uniform sample of n tanks includes the highest-numbered tank; otherwise, the MLE will be off by the difference between M and the maximum value observed in the sample.

3 Part C

Using different values of M and n, simulate different instances of the German Tank Problem, visualize the mean and variance of these three estimators ($\hat{M}_{MLE}$, $\hat{M}_{MEAN}$, $\hat{M}_{MVU}$) as a function of M and n (i.e., fix M to, say 100, 1000, 10K etc., and for each M, plot the mean and variance of the estimators as a function of n - you can compute the mean and variance by repeating the simulation multiple times for each combination of M and n). Discuss the differences between the estimators in terms of their bias and variance.

—

I performed simulations of the German Tank Problem for each of these M values: 100, 1000, 10000. For each value, I varied n from 5% to 95% of each population, in 5% increments. For each (M, n) combination, the three estimators were computed across 1000 simulations and mean and variance were collected across the simulations.

Figure 2 illustrates all of the code related to computing each of the three estimators:

```
def mle_M(tank_ids: list) -> int:
    """
    Computes the MLE for the total number of tanks.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: Maximum likelihood estimate of total tanks
    :rtype: int
    """
    return max(tank_ids)

def mean_M(tank_ids: list) -> float:
    """
    Computes the MEAN estimator for the total number of tanks.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: MEAN estimate of total tanks
    :rtype: float
    """
    return 2 * np.mean(tank_ids) - 1

def mvu_M(tank_ids: list) -> float:
    """
    Computes the MVU estimator for the total number of tanks.

    :param tank_ids: List of observed tank IDs
    :type tank_ids: list
    :returns: MVU estimate of total tanks
    :rtype: float
    """
    n = len(tank_ids)
    return max(tank_ids) * ((n + 1) / n) - 1
```

Figure 2: Estimator functions for $\hat{M}_{MLE}$, $\hat{M}_{MEAN}$, $\hat{M}_{MVU}$.

The simulations were handled by a module containing, alongside tank_capture() from Part A, a function for simulating repeated trials of a specific (M, n) configuration of the German Tank Problem and a function for coordinating all of the different (M, n) configurations, shown below:

```
def simulate_estimators(M: int, n: int, num_trials: int) -> dict:
    """
    Simulates the German Tank Problem multiple times and computes
    all three estimators for each.

    :param M: Total number of tanks
```

```
    :type M: int
    :param n: Number of tanks to capture per sim
    :type n: int
    :param num_trials: Number of simulations
    :type num_trials: int
    :returns: Dictionary with lists of estimates for each estimator
    :rtype: dict
    """
    mle_estimates = []
    mean_estimates = []
    mvu_estimates = []

    for _ in range(num_trials):
        captured = tank_capture(M, n)
        mle_estimates.append(mle_M(captured))
        mean_estimates.append(mean_M(captured))
        mvu_estimates.append(mvu_M(captured))

    return {
        'mle': mle_estimates,
        'mean': mean_estimates,
        'mvu': mvu_estimates
    }

def run_simulations(M_values: list, num_trials: int) -> pd.DataFrame:
    """
    Runs simulations for all (M, n) combinations,
    computing mean and variance of each estimator.

    :param M_values: List of M values to test
    :type M_values: list
    :param num_trials: Number of trials per (M, n) combination
    :type num_trials: int
    :returns: DataFrame with simulation results
    :rtype: pd.DataFrame
    """
    n_percentages = [p * 0.01 for p in range(5, 100, 5)]
    results = []

    for M in M_values:
        n_values = [max(1, int(M * p)) for p in n_percentages]

        for n in n_values:
            estimates = simulate_estimators(M, n, num_trials)

            results.append({
                'M': M,
                'n': n,
                'mle_mean': np.mean(estimates['mle']),
                'mle_var': np.var(estimates['mle']),
                'mean_mean': np.mean(estimates['mean']),
                'mean_var': np.var(estimates['mean']),
                'mvu_mean': np.mean(estimates['mvu']),
                'mvu_var': np.var(estimates['mvu'])
            })

    df = pd.DataFrame(results)
    return df
```

Figure 3: Simulation functions for comparing the three estimators.

Below is the visualization code used to generate the comparison plots for estimator mean and variance:

```
def plot_estimator_means(results_df: pd.DataFrame, output_dir: str):
    """
    Plots the mean of each estimator as a function of n, for each M value.

    :param results_df: DataFrame of estimator means and variances
    :type results_df: pd.DataFrame
    :param output_dir: Where to save the figures
    :type output_dir: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")

    M_values = results_df['M'].unique()

    for M in M_values:
        df = results_df[results_df['M'] == M]

        plt.figure(figsize=(12, 7))

        ax = plt.gca()
        ax.plot(df['n'], df['mle_mean'],
                marker='o', markersize=4, label='MLE', linewidth=2, alpha=0.8)
        ax.plot(df['n'], df['mean_mean'],
                marker='o', markersize=4, label='MEAN', linewidth=2, alpha=0.8)
        ax.plot(df['n'], df['mvu_mean'],
                marker='o', markersize=4, label='MVU', linewidth=2, alpha=0.8)
        ax.axhline(y=M, color='black', linestyle='--', linewidth=1.5,
                   label=f'True M={M}')

        ax.set_xlabel('Sample Size n', fontsize=14, labelpad=15)
        ax.set_ylabel('Mean Estimate', fontsize=14, labelpad=15)
        ax.set_title(f'Mean of Estimators vs Sample Size (M={M})', fontsize=16, pad=20)

        legend = ax.legend(fontsize=12, loc='best')
        for text in legend.get_texts():
            text.set_fontsize(11)

        plt.tight_layout()
        os.makedirs(output_dir, exist_ok=True)
        plt.savefig(f'{output_dir}/mean_comparison_M{M}.png', bbox_inches='tight', dpi=300)

def plot_estimator_variances(results_df: pd.DataFrame, output_dir: str = '../figures'):
    """
    Plots the variance of each estimator as a function of n, for each M value.

    :param results_df: DataFrame from estimator means and variances
    :type results_df: pd.DataFrame
    :param output_dir: Where to save the figures
    :type output_dir: str
    """
    sns.set_style("whitegrid")
    sns.set_palette("muted")

    M_values = results_df['M'].unique()

    for M in M_values:
        df = results_df[results_df['M'] == M]

        plt.figure(figsize=(12, 7))

        ax = plt.gca()
```

```
ax.plot(df['n'], df['mle_var'],
        marker='d', markersize=8, label='MLE', linewidth=2, alpha=0.8)
ax.plot(df['n'], df['mean_var'],
        marker='o', markersize=4, label='MEAN', linewidth=2, alpha=0.8)
ax.plot(df['n'], df['mvu_var'],
        marker='o', markersize=4, label='MVU', linewidth=2, alpha=0.8)

ax.set_xlabel('Sample Size n', fontsize=14, labelpad=15)
ax.set_ylabel('Variance', fontsize=14, labelpad=15)
ax.set_title(f'Variance of Estimators vs Sample Size (M={M})', fontsize=16, pad=20)

legend = ax.legend(fontsize=12, loc='best')
for text in legend.get_texts():
    text.set_fontsize(11)

plt.tight_layout()
os.makedirs(output_dir, exist_ok=True)
plt.savefig(f'{output_dir}/variance_comparison_M{M}.png', bbox_inches='tight', dpi=300)
```

Figure 4: Visualization functions for estimator comparison.

You'll notice that in the variance figures, I chose to make the markers associated with the MLE plots larger and in the shape of diamonds because at high population scales, the plots of MLE (blue) and MVU (green) variance are so similar to each other, relative to MEAN, that they almost perfectly overlap each other visually. This was intended to make sure the reader can still recognize the individual plots in the figures.

Figures 5-7 show the mean of each estimator as a function of sample size n for $M = 100, 1000, 10000$.

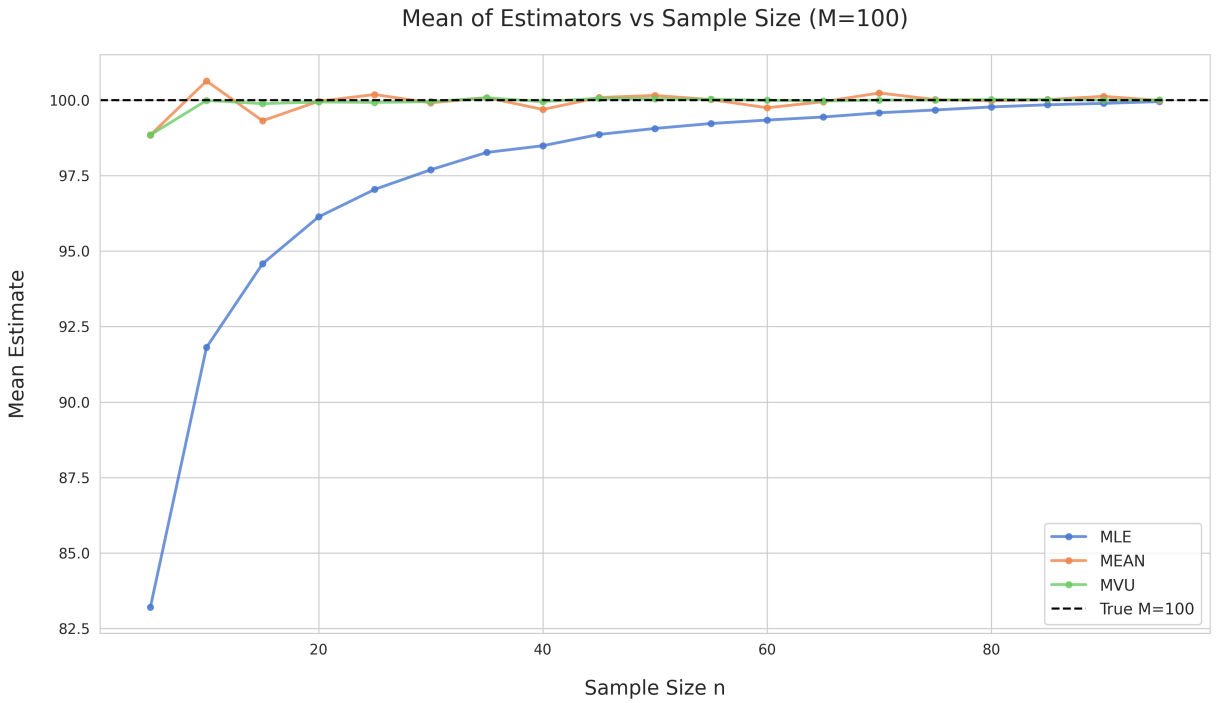

Figure 5: Mean of estimators vs sample size for $M = 100$.

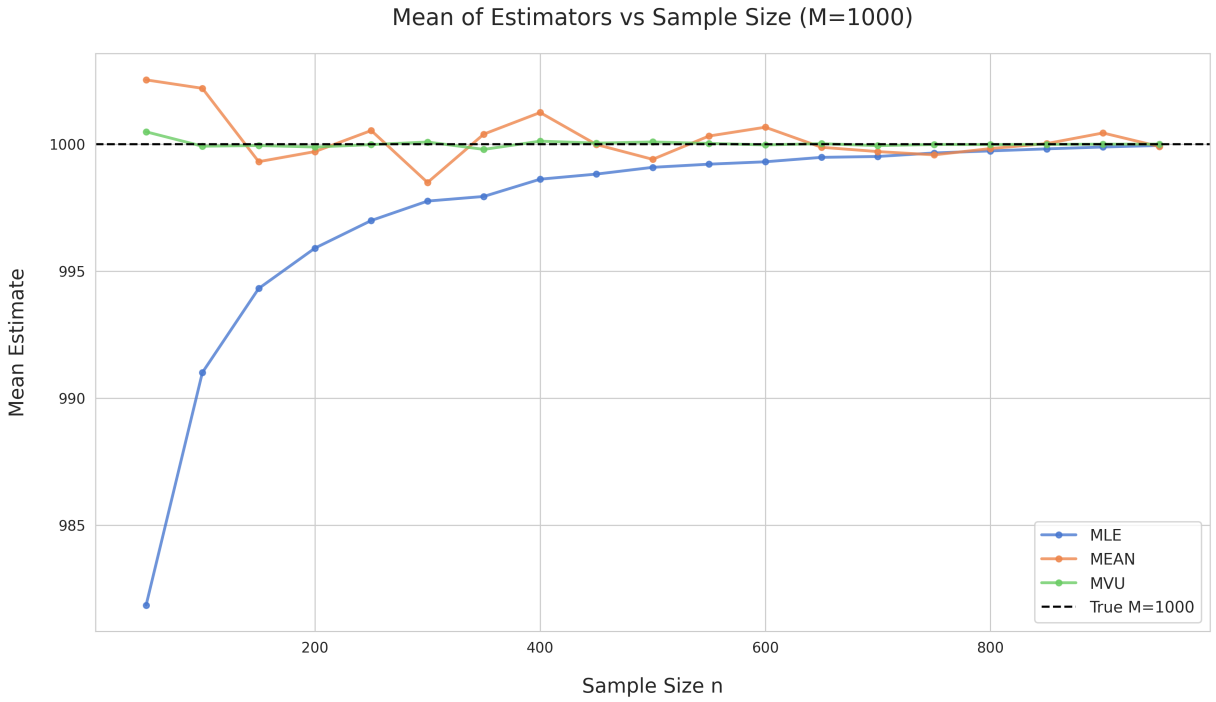

Figure 6: Mean of estimators vs sample size for $M = 1000$.

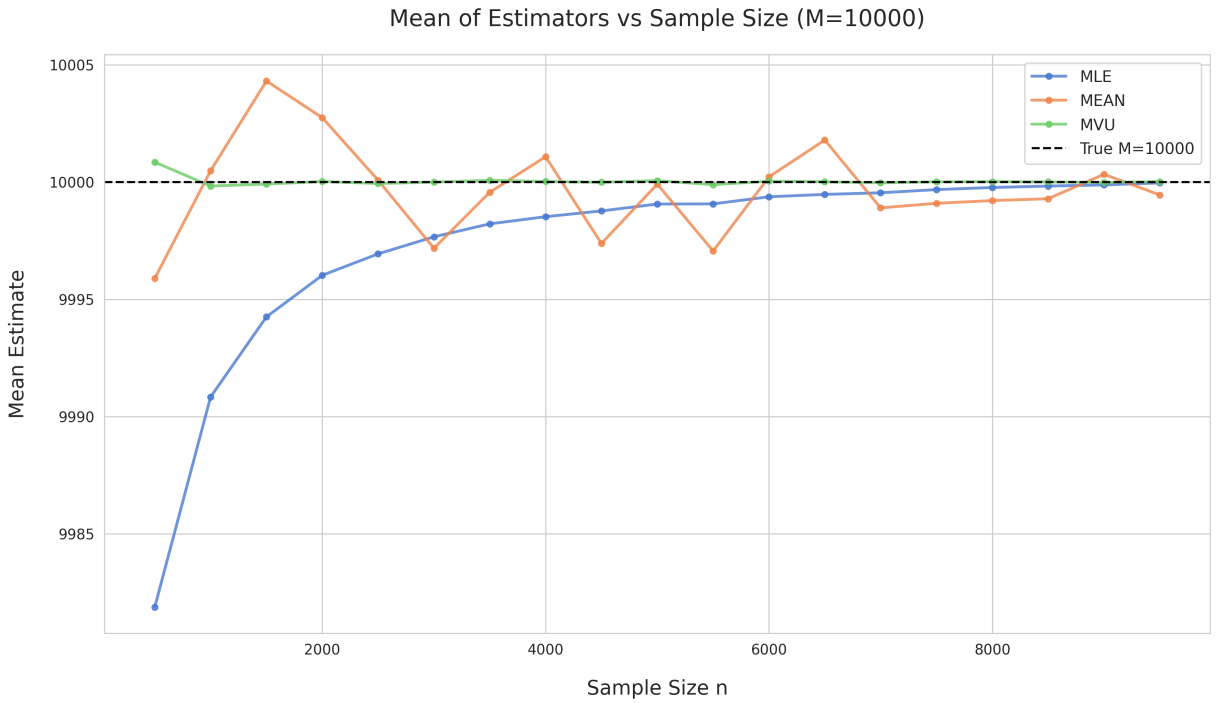

Figure 7: Mean of estimators vs sample size for $M = 10000$.

Figures 8-10 show the variance of each estimator as a function of sample size n for the same values of M.

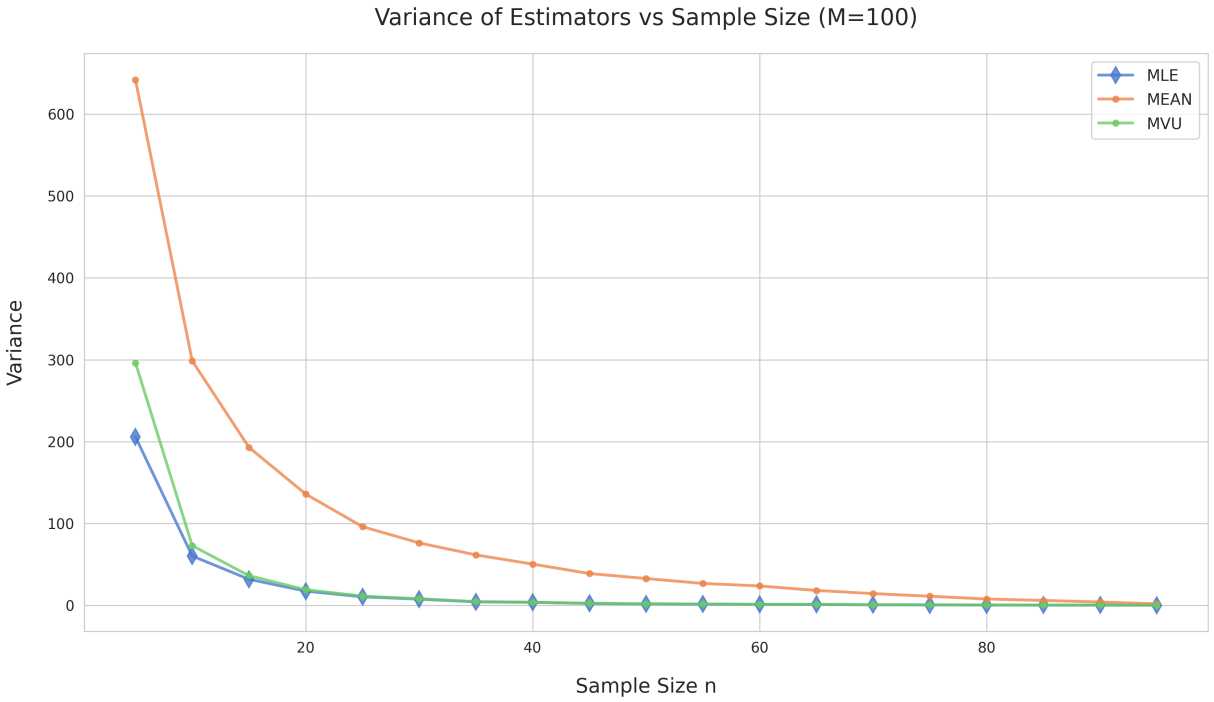

Figure 8: Variance of estimators vs sample size for $M = 100$.

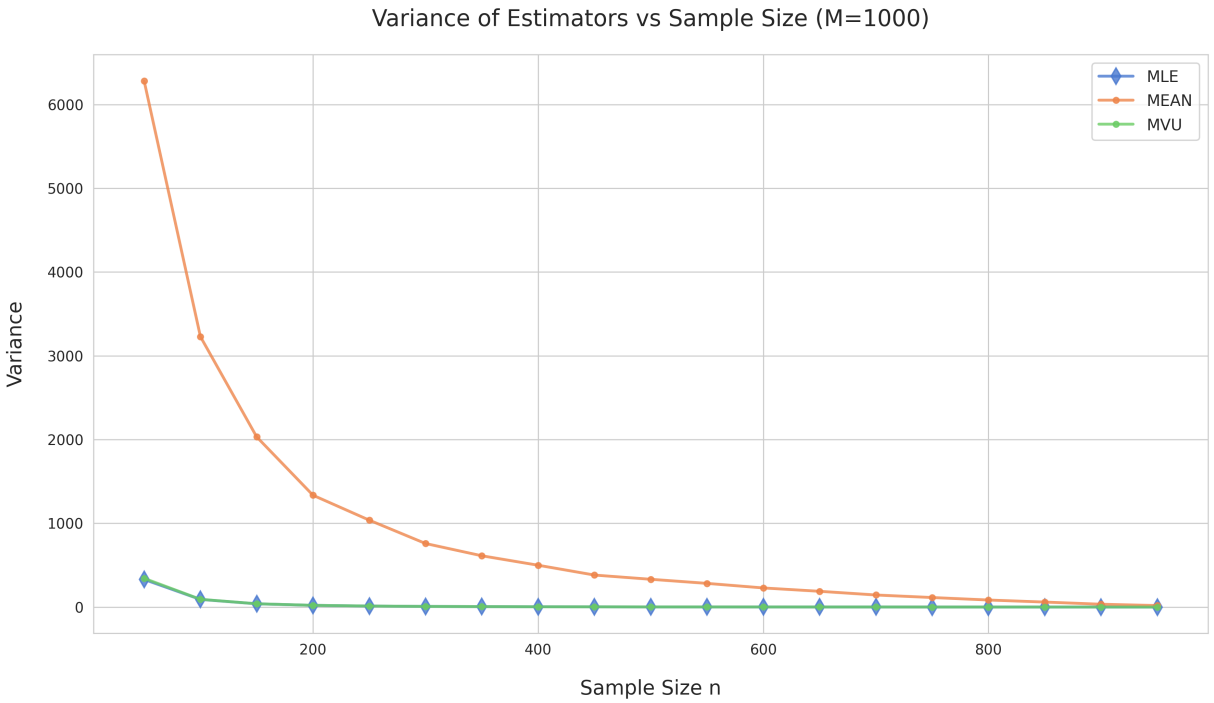

Figure 9: Variance of estimators vs sample size for $M = 1000$.

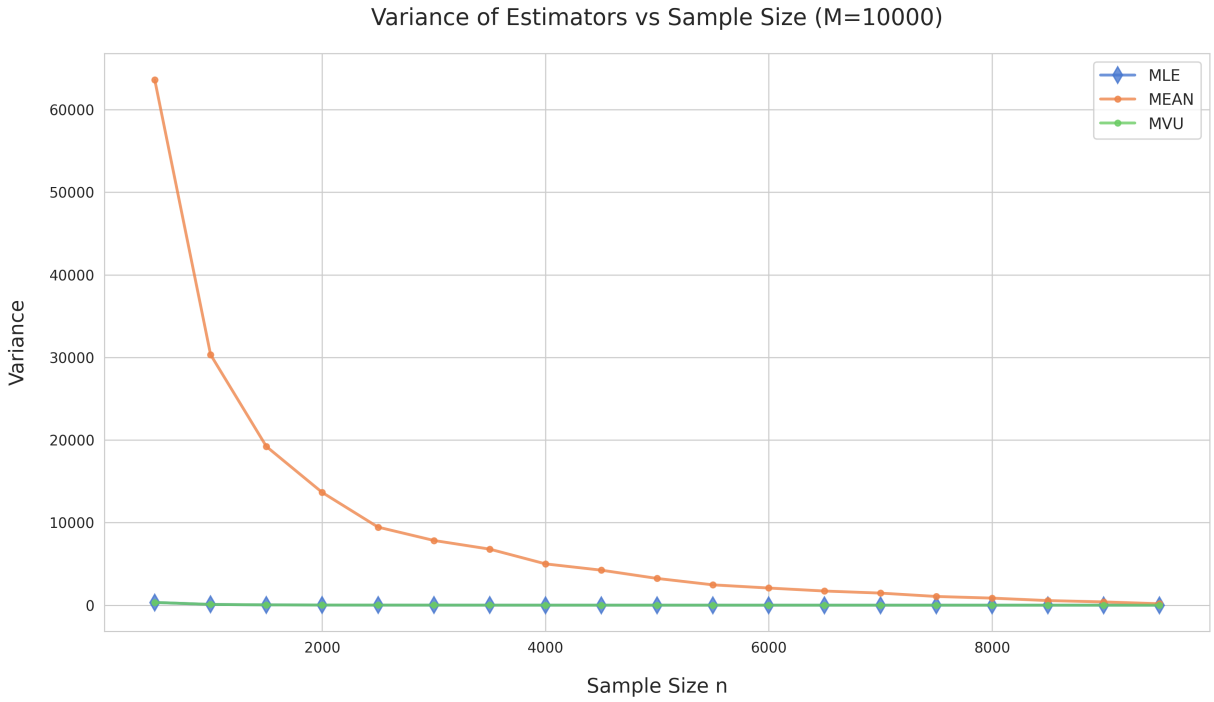

Figure 10: Variance of estimators vs sample size for $M = 10000$.

The plots of estimator mean in each of the three figures clearly illustrate the unique bias of the MLE for this German Tank Problem, where it consistently underestimates the true value of M. This estimate then becomes more and more accurate to the true population as n increases, approaching the dashed line representing true M in the figure. We can think of that behavior intuitively as the uniform, random sampling having more opportunities to observe a value closer to the actual maximum ID present for the population.

The correction factor applied for MVU yields a very satisfying outcome, as the plot quickly converges to the true parameter value, riding the dashed line at true M across the majority of the sample sizes. The MVU estimate is calibrated exactly as it needs to be to adjust for the structural underestimation present in the MLE, because the correction factor $\frac{n+1}{n}$ is literally the inverse of the factor underlying the undervaluation of the MLE, expressed in its expectation and is what we observed in the calculation of MLE bias in Part B: $E[\hat{M}_{MLE}] - M = \frac{n}{n+1}(M+1) - M$. In the case of MVU, the expected value simplifies to $E[\hat{M}_{MVU}] - M = \frac{n+1}{n}\frac{n}{n+1}(M+1) - M = (M+1) - M$. I'm assuming that similar to what we discussed in lecture last week, the convention of the $+1$ factor in this final expression is sometimes omitted sometimes not, and for large M, is very insignificant, though regardless the expected value of MVU was provided by the professor as M. In any case, the MVU is corrected by exactly what is necessary to fully address the negative bias present in the MLE.

MEAN is also unbiased like MVU, however it's visually clear from the estimator mean figures that the underlying mechanism that makes MEAN unbiased is not nearly as well-calibrated for smaller n. MEAN is unbiased in the sense that the sample mean of the observations will approximate the population mean and will become more representative of the population mean for greater numbers of samples.

That being said, this approximation is more sensitive to the uniform, random sampling for small n, as opposed to the careful correction of MVU with respect to sample size, $\frac{n+1}{n}$. For this reason, we can see in the variance figures for any of the M values that as n decreases, the MEAN estimate skyrockets in variance compared to the MVU estimate because the estimate's stability is dependent on the random chance that the individual sets of observations over the repeated simulations happen to yield similar results.

Regarding the variance of the MLE, we can now verify visually that variance is quite low in each of the cases of M and across the different sample sizes n. While structurally biased, the design of MLE yields consistently close estimates given some set number of chances to sample a value close to M. In other words, its inconsistency in estimating M depends only on how many chances the sampling is given to observe values close to M, and so as n increases, the estimator converges quite tightly to M.